Hoisting

Why one call works and the other crashes

snippet-1.js

```
1 greet();
2 function greet() {
3   console.log("Hello");
4 }
```

output **Hello**

snippet-2.js

```
1 greet();
2 const greet = () => {
3   console.log("Hello");
4 };
```

output **ReferenceError**

What hoisting does

01 Scan first
Before running, JS scans the code and sets up memory for every declaration.

02 function
Hoisted completely: name AND body. You can call it before its line.

03 let / const
Hoisted, but locked in the Temporal Dead Zone (TDZ) until their line runs.

04 var
Hoisted and set to `undefined`. No crash, but no value yet.

Step by step (snippet 2)

1. JS scans the code and finds `const greet`.
2. It reserves `greet` in memory, but keeps it locked (TDZ).
3. Line 1 runs `greet()` while it is still locked.
4. Result: ReferenceError: Cannot access 'greet' before initialization.

var vs let vs function

What you get if you use it before its line

before-the-line.js

```
1 console.log(a); // undefined
2 var a = 5;
3 console.log(b); // ReferenceError
4 let b = 5;
5 say(); // TypeError: not a function
6 var say = function () {};
```

declaration	hoisted as	used before its line
`function f() {}`	full function	works
`var x`	undefined	undefined
`let / const / class`	locked (TDZ)	ReferenceError
`var f = () => {}`	undefined	TypeError

Traps to mention

- **Arrow functions follow their variable.** `const` arrow = TDZ, `var` arrow = undefined.
- **TDZ is about time, not position.** A function can use a const declared below it, if it is called after that line runs.
- **Declare before you use.** Hoisting explains bugs; it is not a style to rely on.

Your 20-second answer

"Before running code, JavaScript hoists declarations into memory. Function declarations are hoisted fully, so they can be called early. let and const are hoisted but stay in the Temporal Dead Zone until their line runs, so using them early throws a ReferenceError. var is hoisted as undefined."